Documentación Técnica

Sistema de Reservas y Boletos Aéreos sobre Apache Cassandra (aerolinea_db)

Nombre: Luis Pedro Pérez Monzón

Carné: 201901271

---

1. Introducción

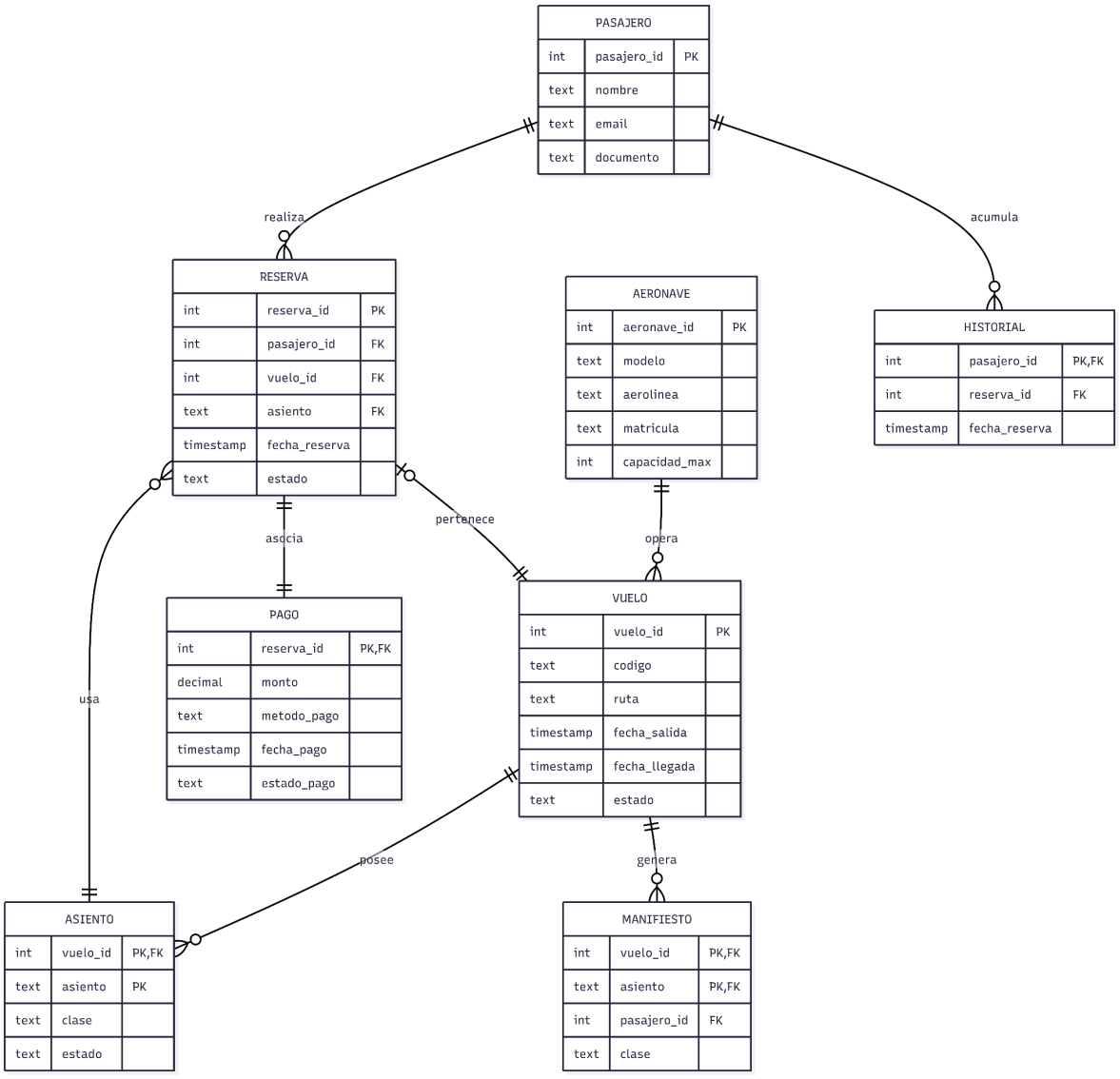
Se implementó la base de datos distribuida de una aerolínea en Apache Cassandra (clúster de 3 nodos en Docker), siguiendo el enfoque Query-Driven Modeling: cada tabla está diseñada para resolver una sola consulta de negocio con una lectura de partición, sin `JOIN`, sin `ALLOW FILTERING` y sin `COUNT` sobre tablas base en tiempo de lectura (las agregaciones viven en la BD).
Datos cargados y verificados:

Indicador	**Valor**
Reservas (batch)	164,234
Pagos por reserva	164,234
Historial de pasajeros	164,234
Manifiesto de vuelo (detalle)	151,244
Q1 — conteo de asientos por (vuelo, clase)	3,600
Q4 — ocupación por (ruta, mes, vuelo)	1,200
Q5 — ingresos por (mes, vuelo)	3,318
Nodos	3 (`cassandra1/2/3`)
Replication factor	3
Consistency operativo	QUORUM
Estrategia de replicación	NetworkTopologyStrategy (dc1)

2. Modelo conceptual (entidad-relación)

Entidades: Aeronave, Vuelo, Asiento, Pasajero, Reserva, Pago e Historial, con las reglas:

- Una aeronave opera muchos vuelos (1 : N).
- Un vuelo posee muchos asientos, cada uno con clase y estado (1 : N).
- Un pasajero realiza muchas reservas (1 : N).
- Cada reserva pertenece a un vuelo y a un asiento (N : 1), y se asocia a un pago (1 : 1).
- Cada cambio de estado deja un registro en el historial (1 : N).
- Un asiento de un vuelo es exclusivo para un pasajero a la vez.



Entidad	Atributos
Aeronave	aeronave_id, modelo, aerolinea, matricula, capacidad_max
Pasajero	pasajero_id, nombre, email, documento, telefono, nacionalidad
Vuelo	vuelo_id, codigo_vuelo, aeronave_id, vuelo, origen, destino, ruta, fecha_salida, fecha_llegada, estado
Asiento	asiento, clase, estado
Reserva	reserva_id, pasajero_id, vuelo_id, asiento, fecha_reserva, estado
Pago	reserva_id, monto, metodo_pago, fecha_pago, estado_pago
Historial	pasajero_id, fecha_reserva, reserva_id, vuelo_id, estado_reserva, estado_pago, ...

---

3. Fundamentos del diseño en Cassandra

En Cassandra no existen los JOINs y `ALLOW FILTERING` es un anti-patrón. Por ello se aplica el modelado orientado a consultas: "una tabla por patrón de acceso". La clave se elige para que la consulta complete en una sola partición, y las columnas de clustering proveen orden y rangos nativos. Las consultas del negocio (Q1–Q5) se definieron primero; luego se diseñó una tabla para cada una.

3.1 Keyspace y replicación

```
CREATE KEYSPACE aerolinea_db
WITH replication = {
  'class'              : 'NetworkTopologyStrategy',
  'dc1'                : 3
};
```

- NetworkTopologyStrategy (recomendada por DataStax; soporta múltiples datacenters y RF por DC). En producción no se usa `SimpleStrategy`.

- RF = 3: cada fila se replica en los 3 nodos del DC; permite QUORUM (2 de 3) y tolera la caída de un nodo.

---

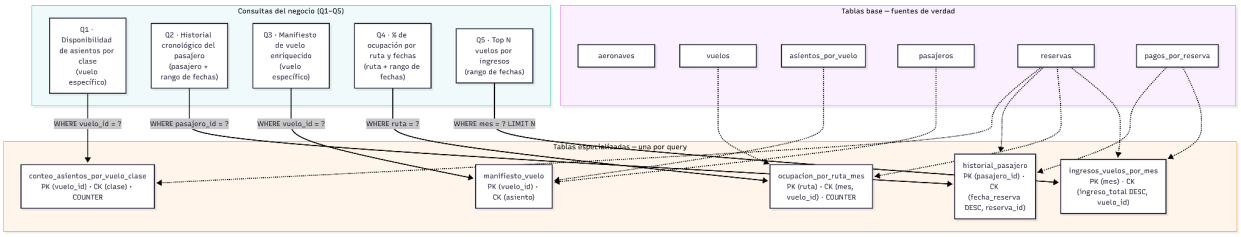
4. Modelo lógico (query-driven) y CQL

Tablas base (fuentes de verdad) y especializadas (una por consulta):

Tabla	**Propósito**	**Clave de partición**	**Clustering**
`aeronaves`	Parque aéreo	aeronave_id	—
`vuelos`	Programación (denormaliza ruta, capacidad)	vuelo_id	—
`asientos_por_vuelo`	Asientos de cada vuelo	vuelo_id	asiento
`pasajeros`	Directorio	pasajero_id	—
`reservas`	Fuente de verdad de reservas	reserva_id	—
`pagos_por_reserva`	Pago de cada reserva	reserva_id	—
`historial_pasajero`	Q2 — historial por pasajero	pasajero_id	(fecha_reserva DESC, reserva_id)
`manifiesto_vuelo`	Q3 — manifiesto del vuelo	vuelo_id	asiento
`conteo_asientos_por_vuelo_clase`	Q1 — COUNTER	vuelo_id	clase
`ocupacion_por_ruta_mes`	Q4 — COUNTER	ruta	(mes, vuelo_id)
`ingresos_vuelos_por_mes`	Q5 — DETALLE top	mes	(ingreso_total DESC, vuelo_id)

4.1 Mapa query-driven (consultas → tablas)

El siguiente diagrama resume el enfoque query-driven: cada consulta del negocio (Q1–Q5) se resuelve con una única tabla especializada cuya clave de partición coincide con el `WHERE` de la query, y cuyas clustering keys proveen orden y rangos nativos. Las tablas especializadas se denormalizan desde las tablas base (indicado con flechas discontinuas) para eliminar JOINs:



4.2 Tabla base `reservas` (fuente de verdad)

```
CREATE TABLE aerolinea_db.reservas (
    reserva_id      int PRIMARY KEY,
    pasajero_id     int,
    vuelo_id        int,
    asiento         text,
    fecha_reserva   timestamp,
    estado          text
);
```

4.3 Tabla base `pagos_por_reserva` + `historial_pasajero` (Q2)

Cada reserva tiene **un pago** y se denormaliza su **historial** por pasajero para responder Q2 (historial cronológico del pasajero con estado de reserva y de pago) en **una sola partición**:

```
CREATE TABLE aerolinea_db.historial_pasajero (
    pasajero_id     int,
    fecha_reserva   timestamp,
    reserva_id      int,
    vuelo_id        int,
    codigo_vuelo    text,
    origen          text,
    destino         text,
    fecha_salida    timestamp,
    asiento         text,
    clase           text,
    estado_reserva  text,
```

```
    estado_pago     text,
    monto           decimal,
    metodo_pago     text,
    fecha_pago      timestamp,
    PRIMARY KEY (pasajero_id, fecha_reserva, reserva_id)
) WITH CLUSTERING ORDER BY (fecha_reserva DESC, reserva_id DESC);
```

- Partition key = pasajero_id → todo el historial del pasajero es una partición; consulta por rango de fechas = rango de clustering sin `ALLOW FILTERING`.
- Clustering (fecha_reserva, reserva_id) DESC → orden cronológico inverso nativo.

4.4 Tabla base `manifiesto_vuelo` (Q3)

Manifiesto del vuelo (asiento, pasajero, documento, clase, estados): partition key `vuelo_id`, clustering `asiento` (ordena el manifiesto por número de asiento).

---

5. Tablas especializadas (agregación por consulta)

5.1 Q1 — `conteo_asientos_por_vuelo_clase` (COUNTER)

```
CREATE TABLE aerolinea_db.conteo_asientos_por_vuelo_clase (
    vuelo_id     int,
    clase        text,
    disponibles  counter,
    ocupados     counter,
    bloqueados   counter,
    PRIMARY KEY (vuelo_id, clase)
);
```

- Partition key = vuelo_id → todas las clases del vuelo en una partición → Q1 (`WHERE vuelo_id = ?`) es una sola lectura.
- Clustering por `clase` para ordenar el desglose.
- COUNTER: el conteo cambia con cada reserva/cancelación; en lugar de recontar 151,244 asientos en lectura, se incrementa con `UPDATE ... SET disponibles = disponibles + ?`.

5.2 Q4 — `ocupacion_por_ruta_mes` (COUNTER)

```
CREATE TABLE aerolinea_db.ocupacion_por_ruta_mes (
    ruta                 text,
    mes                  text,   -- "YYYY-MM"
    vuelo_id             int,
    reservas_confirmadas counter,
    capacidad            counter,
    PRIMARY KEY (ruta, mes, vuelo_id)
) WITH CLUSTERING ORDER BY (mes ASC);
```

- Partition key = ruta → todos los vuelos de la ruta en una partición.
- Clustering `(mes, vuelo_id)` permite rangos de meses; la ocupación % = `reservas_confirmadas / capacidad` con dos counters de la misma fila.

5.3 Q5 — `ingresos_vuelos_por_mes` (DETALLE, clustering DESC)

```
CREATE TABLE aerolinea_db.ingresos_vuelos_por_mes (
    mes            text,
    ingreso_total  decimal,
    vuelo_id       int,
    codigo_vuelo   text,
    PRIMARY KEY (mes, ingreso_total, vuelo_id)
) WITH CLUSTERING ORDER BY (ingreso_total DESC, vuelo_id ASC);
```

- Partition key = mes; clustering `ingreso_total DESC` → el Top-N ya está ordenado físicamente: `SELECT ... WHERE mes = ? LIMIT 10` devuelve el Top-10 sin ORDER BY en el cliente.
- Es una tabla de DETALLE (no COUNTER, porque `ingreso_total` es clave de clustering): se reconstruye con `TRUNCATE + INSERT` agregado desde los pagos confirmados, idéntico al proceso de carga (`load_data.py`) y idempotente.

5.4 TTL — eliminación automática de reservas expiradas

Se creó la tabla `reservas_expiradas` con `default_time_to_live = 7,776,000` s (90 días): recibe solo las reservas que pierden vigencia (canceladas o pendientes sin pago) y Cassandra las elimina solas al expirar, sin intervención ni job de limpieza. Las reservas *confirmadas y pagadas* persisten en `reservas` e `historial_pasajero` para historial y auditoría.

```
CREATE TABLE aerolinea_db.reservas_expiradas (
    reserva_id     int PRIMARY KEY,
    pasajero_id    int,
    vuelo_id       int,
    asiento        text,
    fecha_reserva  timestamp,
    estado         text           -- cancelada | pendiente
) WITH default_time_to_live = 7776000;  -- 90 días en segundos
```

Carga (Fase 4): el batch de cada reserva agrega la escritura en `reservas_expiradas` cuando `estado ∈ {cancelada, pendiente}` (`carga/load_data.py`). La reconstrucción idempotente quedó a cargo de `carga/verificar_reparar.py` (TRUNCATE + INSERT vuelve a fijar el TTL).

Verificación en vivo (el TTL cuenta hacia atrás desde 7,776,000 s):

```
SELECT reserva_id, estado, TTL(estado) AS ttl_restante_seg
FROM aerolinea_db.reservas_expiradas LIMIT 5;
```

reserva_id	estado	ttl_restante_seg
62693	cancelada	7,775,542
39433	cancelada	7,775,540
48451	cancelada	7,775,541
127820	cancelada	7,775,552
9892	cancelada	7,775,535

- **Filas con TTL:** 13,748 (total de `reservas_expiradas`).
- **Interpretación:** el TTL decrece en vivo (~90 días restantes); al llegar a `0` la fila desaparece con el siguiente compactación, sin consultas programadas ni `DELETE` manuales.

---

6. Configuración del clúster y consistencia

Parámetro	Valor	Justificación
Nodos	3 (cassandra1/2/3)	Mínimo para RF=3 y QUORUM
Replication	NetworkTopologyStrategy, RF=3	DC-aware; réplica completa
Consistency base	QUORUM	Equilibrio consistencia/disponibilidad; tolera 1 nodo caído
Consistency de escritura	QUORUM	Confirma en 2 de 3 réplicas
puertos CQL	9042 / 9142 / 9242	Un puerto por nodo

3.6 Distribución verificada entre los 3 nodos

`nodetool status` — los 3 nodos en UN, carga repartida de forma casi uniforme (~33 MiB por nodo):

Datacenter: dc1

-- Address Load Tokens Owns (effective) Host ID Rack

UN 172.18.0.4 33.46 MiB 16 100.0% 22b6f0f2-... rack1

UN 172.18.0.3 33.65 MiB 16 100.0% 8dabbf0b-... rack1

UN 172.18.0.2 33.88 MiB 16 100.0% 21421e21-... rack1

`nodetool getendpoints` con RF=3 cada partición tiene réplica en los 3 nodos (se muestra en qué nodos vive una partición real de cada tabla):

Tabla	Partición	Réplicas (endpoints)
reservas	1	172.18.0.2 (coord), 172.18.0.4, 172.18.0.3
historial_pasajero	4317	172.18.0.3, 172.18.0.2, 172.18.0.4
manifiesto_vuelo	769	172.18.0.3, 172.18.0.2, 172.18.0.4

Tabla	Partición	Réplicas (endpoints)
conteo_asientos_por_vuelo_clase	769	172.18.0.3, 172.18.0.2, 172.18.0.4
ocupacion_por_ruta_mes	GUA-PTY	172.18.0.4, 172.18.0.3, 172.18.0.2
ingresos_vuelos_por_mes	2026-07	172.18.0.2, 172.18.0.4, 172.18.0.3

`nodetool tablestats aerolinea_db.reservas` (espacio por réplica):

Nodo	Espacio usado
cassandra1	7,061,387 B
cassandra2	7,276,524 B
cassandra3	7,267,226 B

Interpretación: los 16 vnodes por nodo reparten la tabla `reservas` en los 3 peers (~7 MiB por réplica) y cada partición queda replicada 3 veces (RF=3). Esto garantiza que con CL=QUORUM una consulta sigue resolviéndose con 2 de 3 réplicas incluso si cae un nodo, y que la pérdida de un nodo no deja datos huérfanos (todos tienen réplica completa).

---

7. Consultas CQL implementadas

Las 5 consultas Q1–Q5 se resuelven las tablas especializadas. Ver ejemplos completos y parámetros en el Manual de Usuario (`Manual_Usuario.md`). Todas operan sin ALLOW FILTERING (el filtro es siempre la clave de partición, y los rangos son clustering keys).

7.1 Latencias medidas (ms, CL=QUORUM, 100 iteraciones)

Consulta	Tabla	Filas	Promedio	p50	p95	Mínimo	Máximo
Q1 · Disponibilidad por clase (vuelo 769)	conteo_asientos_por_vuelo_clase	3	4.28	3.50	8.01	2.01	13.45
Q2 · Historial cronológico (pasajero 4317, 2026)	historial_pasajero	3	4.62	4.32	7.71	2.63	11.12

Consulta	Tabla	Filas	Promedio	p50	p95	Mínimo	Máximo
Q3 · Manifiesto enriquecido (vuelo 769)	manifiesto_vuelo	116	9.17	8.13	18.52	3.93	23.52
Q4 · Ocupación por ruta/meses (GUA-PTY, 2026-01)	ocupacion_por_ruta_mes	25	6.05	5.31	12.47	2.80	15.18
Q5 · Top-5 por ingresos (2026-07)	ingresos_vuelos_por_mes	5	5.98	5.44	11.37	2.44	14.49

Todas las consultas resuelven en < 10 ms de promedio sobre 164,234 reservas (datos en caliente, key cache caliente). Q3 es la de mayor p95 porque devuelve 116 filas (manifiesto completo).

7.2 Plan de ejecución (TRACING) — patrón por consulta

El trace de cada consulta confirma el mismo patrón eficiente:

1. `Executing single-partition query on <tabla>` → una sola partición (la clave de partición coincide exactamente con el `WHERE`; sin ALLOW FILTERING ni range scan).
2. Lectura en la réplica coordinadora (`SKIPPED N/M sstables`, `Key cache hit`, `Read K live rows`).
3. `Sending READ_REQ message` a las réplicas → QUORUM lee 2 de 3 réplicas y reconcilia (compara digest de la 2.ª réplica).

Trace real de Q1 (coordinador 172.18.0.3, duración total 17,984 µs):

```
  coordenador=172.18.0.3 duración=17984 µs
        2793µs   172.18.0.3  Executing single-partition query on
conteo_asientos_por_vuelo_clase
        3056µs   172.18.0.3  Skipped 0/0 non-slice-intersecting sstables
        3249µs   172.18.0.3  Read 3 live rows and 0 tombstone cells
        3991µs   172.18.0.3  reading digest from /172.18.0.2:7000
        ...
       13822µs   172.18.0.3  READ_RSP message received from /172.18.0.2:7000   (réplica 1)
       14463µs   172.18.0.3  Processing response from /172.18.0.2:7000
       14649µs   172.18.0.3  READ_RSP message received from /172.18.0.4:7000   (réplica 2)
```

Query	Traza clave (actividad dominante)
Q1, Q2, Q3, Q4, Q5	`single-partition query` + `Read K live rows` + `READ_REQ/READ_RSP` a 2 réplicas (QUORUM)

Interpretación: ninguna consulta invoca `ALLOW FILTERING`, `JOIN` ni `ORDER BY` en el cliente; el orden y los rangos vienen de las clustering keys (Q2 `fecha_reserva DESC`, Q3 `asiento`, Q5 `ingreso_total DESC`), y las agregaciones Q1/Q4 viven en contadores de la BD.

---

8. Pruebas de tolerancia a fallos (métricas por Consistency Level)

Se simuló la caída del nodo 2 (`cassandra2`, puerto 9142) y se midió, con 2 de 3 nodos vivos, la misma consulta con diferentes Consistency Levels (10 intentos cada una):

Consistency Level	Intentos	Éxitos	Fallos	Disponibilidad
`ONE`	10	10	0	100 %
`QUORUM`	10	10	0	100 %
`ALL`	10	0	10	0 % — `UnavailableException`: *required 3, alive 2*

Interpretación: con RF=3 y CL=QUORUM el clúster tolera la caída de un nodo manteniendo lecturas correctas y disponibles. `ALL` queda indisponible ante la caída de una sola réplica (esperado), y `ONE` prioriza disponibilidad con menor garantía de consistencia. Latencia comparativa (clúster sano): `ONE < QUORUM < ALL`; QUORUM añade el round-trip de 2 réplicas.

Recuperación

Al reiniciar `cassandra2`, `nodetool status` lo reincorpora como UN (Up/Normal) en el anillo; la replicación con RF=3 y el mecanismo de hinted handoff re-sincronizan los datos sin intervención manual — el clúster recupera consistencia total automáticamente.

---

9. Verificación y reparación (idempotente)

`verificar_reparar.py`:

- Cuenta filas por tabla y compara contra lo esperado.
- Reconstruye las tablas de agregación Q1/Q4/Q5 (TRUNCATE + INSERT/UPDATE) de forma idempotente (repetir no duplica).
- Corrige faltantes en tablas base re-insertando desde los CSV.
- Es auditado por conteos finales (3,600 / 1,200 / 3,318).

Ejecución de `carga/verificar_reparar.py` (CL=QUORUM): 0 filas faltantes en `reservas`, `pagos_por_reserva` e `historial_pasajero`, y reconstrucción idempotente de agregados:

Tabla	**Conteo**
reservas	164,234
pagos_por_reserva	164,234
reservas_expiradas (TTL 90 días)	13,748
historial_pasajero	164,234
manifiesto_vuelo	151,244
conteo_asientos_por_vuelo_clase	3,600
ocupacion_por_ruta_mes	1,200
ingresos_vuelos_por_mes	3,318

Reproducción (comandos)

```
# 1) Cluster (3 nodos, RF=3, dc1)
docker compose -f docker/docker-compose.yml up -d
docker exec cassandra1 nodetool status          # 3× UN
# 2) Esquema (incluye reservas_expiradas con TTL)
docker exec -i cassandra1 cqlsh < ddl/01_keyspace.cql
docker exec -i cassandra1 cqlsh < ddl/02_schema.cql
# 3) Carga masiva + integridad
.venv/bin/python carga/load_data.py
.venv/bin/python carga/verificar_reparar.py
```

```
# 4) Latencias + plan (TRACE) + distribución
.venv/bin/python pruebas/medir_latencias.py --cl QUORUM --iter 100
# 5) TTL en vivo
docker exec cassandra1 cqlsh -e "SELECT reserva_id, estado, TTL(estado)
  FROM aerolinea_db.reservas_expiradas LIMIT 5;"
```

---

10. Conclusiones

1. El modelado por consulta con tablas denormalizadas responde Q1–Q5 sobre 164,234 reservas en milisegundos, sin JOINs ni ALLOW FILTERING.
2. La clave de partición adecuada (coincide con el `WHERE`) es el factor decisivo: ubica los datos de la consulta en una sola réplica.
3. RF=3 + NetworkTopologyStrategy + CL=QUORUM da el mejor balance de consistencia y disponibilidad y tolera la caída de un nodo, como se verificó en las pruebas de fallo.
4. Las COUNTER (Q1, Q4) trasladan la agregación a la escritura y las tablas de DETALLE (Q5) usan clustering DESC para el Top-N sin ordenar en el cliente.